

5) What is Binding and Binding Time?

- Binding: is an association between an attribute and an entity
(Such as between a variable and its type).
- Binding Time: is the moment in the program's life cycle when this association occurs.

46) Type inference?

is implicit type declarations using context which is the type of the value assigned to the variable in a declaration statement.

example in c#:

```
var sum = 0;    (type; int)
var total = 0.0; (type; float)
var name = "Fred"; (type; string)
```

8) What does it mean when a variable is (Explicit) Heap Dynamic?

Are nameless memory cells that are allocated and deallocated by explicit run-time instructions written by the programmer.

Advantage: Providing Dynamic storage management.
Disadvantage: Inefficient and Unreliable.

19) How is a lexical analyzer a part of a syntax analyzer?

A lexical analyzer server as a front-end to the syntax analyzer.

- Lexical analyzer: acts as an interface between the source code and the rest of the phases of a compiler.
- Syntax analyzer: Takes the input from the lexical analyzer as a token stream.

20) Define actual parameter and formal parameter.

- **Actual parameter:** Is the variables passed while calling a function.
- **Formal parameter:** There are the variables declared in the function, and receive their values when a call to the function is made.

22) What are the differences between lexical and syntax analysis?

- **Simplicity:** Techniques for lexical analysis are less complex than those required for syntax analysis.
- **Efficiency:** Syntax analyzer is more efficient as lexical analyzer requires a significant portion of total completion time
- Lexical analyzer is platform dependent; Syntax analyzer is platform independent
- Lexical analyzer is the first phase of compilation; Syntax analyzer is the second phase of compilation.

23) What is the generic subprogram and overloaded subprogram?

- **Generic subprogram:** Is one whose computation can be done on data of different types and on different calls.
- **Overloaded subprogram:** Is one that has the same name as another subprogram in the same referencing environment.

Define Sentences - A string of a language.

Define Lexemes - The lowest-level syntactic unit.

Define Token - The smallest individual element in a program.

Define Parsing - Is the part of the process of analysing syntax that is referred to as syntax analysis.

A language can be defined in two distinct ways, what are they?

- Recognition

- Generation

What is Language Generator?

A generator is a device used to generate the sentences of a language.

36) There are three approaches to build a lexical analyzer, what are they.

- Write a formal description of the token patterns of the language using a descriptive language related to regular expressions.
- Design a state transition diagram that describes the token patterns of the language and write a program that implements the diagram.
- Design a state transition diagram that describes the token patterns of the language and hard-construct a table-driven implementation of the state diagram.

37) Define Regular grammar.

Finite automata can be designed to recognize members of a class of language.

38) Define Finite automata.

State diagram of the form used for lexical analyzers are representations of a class of mathematical machines

27) What is closure subprogram?

Is a nested subprogram and its referencing environment, which together allow the subprogram to be called from anywhere in a program.

66) Array categories:

- **Static array:** Is one in which the subscript ranges are statically bound and storage allocation is static (done before run-time).
- **Fixed stack-dynamic array:** Is one in which the subscript ranges are statically bound but allocation is done during execution.
- **Stack-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic (during execution).
- **Fixed heap-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic, but they are both fixed after storage is allocated.
- **Heap-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic, and can change any number of times during the array's lifetime.

79) Parameter passing methods:

- **Pass-by-value: (In-Mode)**
 - ❖ The value of the actual parameter is used to initialize the corresponding formal parameter.
- **Pass-by-result: (Out-Mode)**
 - ❖ Just before control is transferred back to the caller, the value of the formal parameter is transmitted back to the actual parameter.
- **Pass-by-value-result: (In-Out-Mode)**
 - ❖ Is a combination of Pass-by-value and Pass-by-result.
- **Pass-by-reference: (In-Out-Mode)**
 - ❖ Passing the memory address of where a variable is stored.
- **Pass-by-name: (In-Out-Mode)**
 - ❖ Is one where the variable name is passed.

57) Fetch-Execute cycle

initialize the program counter

repeat

fetch the instruction pointed by the counter

increment the counter

decode the instruction

execute the instruction

end repeat

33) Define Grammar.

Is a section that discusses the formal language generation mechanism.

34) Define Parse tree.

Represents the syntactic structure of a program.

40) What are the two broad classes.

- **Top-down:** Parse tree is built from roots to leaves.
- **Bottom-up:** Parse tree is built from leaves to roots.

35) Define Ambiguity.

A grammar that generates a sentential form for which there are two or more distinct parse trees.

54) Language Categories:

- ❖ **Imperative:** Is a software development model where functions are coded in every step to solve problems.
- ❖ **Functional:** Main mean of making computations by applying functions to given parameters
- ❖ **Object-Oriented:** A computer programming model that organizes software designs around data.
- ❖ **Markup:** Used to specify the layout of information in web documents.

55) Implementation Methods:

- ❖ **Compiler:** Programs are translated into machine language.
- ❖ **Pure interpretation:** The program is translated into machine code for a particular CPU one instruction at a time and executes immediately
- ❖ **Hybrid Implementation System:** A compromise between compiler and pure interpretation.

44) Variable declarations:

- **Explicit declaration:** Is a program statement used for declaring the types of variables.
- **Implicit declaration:** Is a default mechanism for specifying types of variables (the first appearance of the variable in the program).

45) Storage binding:

- **Allocation:** The process of binding a variable to an available memory cell.
- **Deallocation:** The process of unbinding a variable from a memory cell.

56) Compilation steps:

- I. **Lexical Analysis:** Converts characters in the program into lexical units.
 - Lexical units of a program are identifiers, special words and punctuations symbols.
- II. **Syntax Analysis:** Converts characters in the program into parse tree.
 - There parse trees represent the syntactic structure of the program.
- III. **Semantic Analysis:** Generates intermediate code.
 - Intermediate languages sometimes look very similar to **assembly language** and in fact sometimes are actually **assembly language**.
- IV. **Code Generation:** Generates machine code.

60) What are programming environments?

The collection of tools used in software development.

61) Numeric types:

- **Integer:** whole numbers including 0 and negatives.
- **Floating-point:** A decimal number that represents a signed real number.
- **Complex:** An expansion of complex macros in <complex.h>
- **Decimal:** Fixed-point decimal data type.
- **Boolean:** Is a data type that can be either true or false.
- **Character:** Is a data type that stores a character.

62) String length options:

- **Static length string:** The length is set when the string is created.
- **Dynamic length string:** The length of the string can vary with no maximum.
- **Limited Dynamic length string:** The length of the string can vary, however there is a declared fixed maximum.

63) Enumeration type?

All possible values, which are named constants, are provided in the definition.

58) Program counter?

A register that maintains the address of the next instruction to be executed.

59) What are Programming Macros?

Are commands used to specify that code from another file is being included.

#include #define

72) Iterative statements

The repeated execution of the statement or a group of statements.

- **Counter-controlled loop:** Has a loop variable and a way of specifying the number of repetitions of the loop.
- **Logical-controlled loop:** Repetition control is based on Boolean.
- **User-located loop control mechanism:** Programmer decides when the loop ends and continues.
- **Statement-controlled loop:** Used to perform looping operations until the given condition is true.

73) Unconditional branch statement?

Transfer execution to a specified location in the program.

74) What is a Subprogram?

Subprograms are the fundamental building blocks of programs.

75) What is a Subprogram call?

The explicit request that a specific subprogram be executed.

76) What is a Subprogram header?

Is the first part of the subprogram and serves several purposes such as identifying the name of the subprogram and declare the local variables.

77) Local and global variables?

- **Local variable:** The variables that are defined inside a subprogram.
- **Global variable:** The variables that are defined outside that given function.

78) Subprogram types:

- **Function:** A block of code that only runs when it is called.
- **Procedure:** A collection of statements that define parameterized computations. (Has not return value).

47) Elaboration?

Creates, initializes and runs a task.

48) Define Static scoping.

The scope of the variable can be determined statically (before execution).

49) Define Dynamic scoping.

The scope of the variable can be determined dynamically (during run-time).

50) block-structured language?

Permits an array with adjustable length.

51) What is named constant?

Is a variable that is bounded to a value only once.

52) Define Initialization.

Is the binding of a variable to a value at the time it is bounded to a storage.

53) What is Von Neumann Architecture?

Is a prevalent computer architecture used to design a language.

25. 9.5.2 Implementation Models of Parameter Passing

Consider the following program written in C syntax (only syntax - it is not a C code):

```
void swap(int a, int b) {  
    int temp;  
    temp = a;  
    a = b;  
    b = temp;  
}  
  
void main() {  
    int value = 2, list[5] = {1, 3, 5, 7, 9};  
    swap(value, list[0]);  
    swap(list[0], list[1]);  
    swap(value, list[value]);  
}
```

For each of the following parameter-passing methods, what are all of the values of the variables value and list after each of the three calls to swap?

- a. Passed by value
- b. Passed by reference

- a. 2 1 3 5 7 9
2 1 3 5 7 9
2 1 3 5 7 9
- b. 1 2 3 4 7 9
1 3 2 5 7 9
2 3 1 5 7 9

19. How do parentheses affect the precedence rule? What will be the evaluation order? 7.2.1.3 Parentheses

we can alter the precedence and associativity rules by using parentheses in expressions.

A parenthesized part of an expression has precedence over its adjacent unparenthesized parts.

20. Define functional side effect, important to show examples as well. 7.2.2.1 Side Effects

If one function changes either its parameter or a global variable, **it has side effects**.

Here is a expression: $a + \text{fun}(a)$, if $\text{fun}(a)$ has side effects, when function change a , the order of evaluation of these two operands maybe changed.

21. Explain why operator overloading increases the semantic capability of a language? 7.3 Overloaded Operators

This multiple use of an operator is called operator overloading.

A single operator symbol has more than one meaning

1. Describe lexical and syntax analysis with a few sentences! 4.2&4.3

Lexical analysis: It is essentially a pattern matcher and serves as the front end of a syntax analyzer.

It collects lexemes (collects characters into logical groupings) from a given input string and produce the corresponding tokens (internal codes' categories).

Syntax analysis: the process of analyzing syntax.

It serves two purposes:

1. Make sure the syntax of the input program is syntactically correct.
2. If the grammar input is correct, it will generate a complete parse tree.

6. Describe the ways that aliases can occur with pass-by-reference parameters 5.3.2 Address

Aliases are two or more variables bound to the same memory location.

An example: `int a=0; int &a1=a;` In this case, `a` and `a1` are aliases.

3- what are the design issues for subprograms?

1. What parameter passing methods are provided?
2. Are parameter types checked?
3. Are local variables static or dynamic?
4. Can subprogram definitions appear in other subprogram definitions?
5. What is the referencing environment of a passed subprogram?
6. Can subprograms be overloaded?
7. Are subprograms allowed to be generic?